

# **Introducción a la Programación I**

## **Clase 0: Sistemas y Algoritmos**

# Agenda

- ¿Qué es un algoritmo?
- Algoritmos en la vida cotidiana
- Tipos de algoritmos
- Arquitectura de computadoras
- Análisis y diseño de soluciones
- Instrucciones para la computadora
- Lenguajes de programación — ¿Por qué Python?
- Archivos fuente y cómo se ejecutan
- Cómo funciona el intérprete de Python

# ¿Qué son los algoritmos?

Un **algoritmo** es una secuencia de pasos **definidos y ordenados** para resolver un problema o realizar una tarea específica.

- Los pasos deben ser **claros y precisos**
- Tienen un **inicio** y un **fin**
- Producen un **resultado** a partir de entradas

Si podés describir cómo hacés algo paso a paso, ya estás pensando algorítmicamente.

**¿Lavarse los dientes es un algoritmo?**

# Lavarse los dientes es un algoritmo

1. Agarrar la pasta dental
2. Revisar si queda pasta
3. ¿No tiene? → Buscar una nueva
4. Buscar el cepillo y colocarlo en los dientes
5. Mover de lado a lado
6. ¿Pasaron 2 minutos? → Si no, volver al paso 5
7. Enjuagarse
8. **Fin**

Decisiones, repeticiones, pasos ordenados — todo es parte de un algoritmo.

# Algoritmos en la vida cotidiana

Los algoritmos están en todos lados:

- 📍 **GPS:** encontrar la mejor ruta para llegar a un lugar
- 🔍 **Google:** mostrar los resultados más relevantes
- 🎵 **Spotify / Netflix:** recomendar contenido según tus preferencias
- 🎮 **Videojuegos:** controlar la inteligencia artificial de personajes
- 🏥 **Medicina:** ayudar a diagnosticar enfermedades
- 📦 **Logística:** optimizar la entrega de paquetes

Cuando aprendés a programar, aprendés a crear estas soluciones.

# Tipos de algoritmos

| Tipo         | ¿Qué hace?                                                              |
|--------------|-------------------------------------------------------------------------|
| Búsqueda     | Encontrar un elemento dentro de un conjunto de datos                    |
| Ordenación   | Organizar elementos según algún criterio                                |
| Optimización | Encontrar la mejor solución posible a un problema                       |
| Recursivos   | Resolver un problema repitiéndose sobre versiones más simples del mismo |

# Beneficios de aprender algoritmos

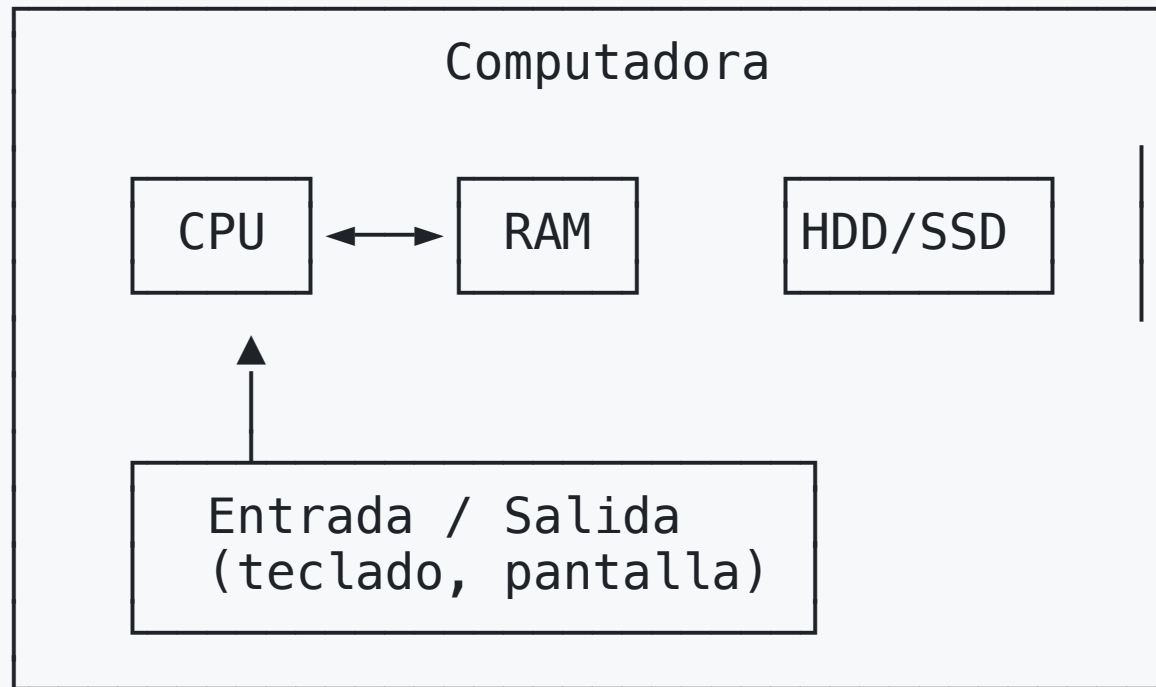
- 🧠 Desarrollar **pensamiento crítico y lógico**
- 💻 Comprender mejor el **mundo digital** que nos rodea
- 🔧 Mejorar la capacidad de **resolver problemas** de forma metódica
- 🏗️ Mejorar la habilidad de **diseñar sistemas y programas**
- 🌐 Base para **cualquier lenguaje** de programación

Los algoritmos son independientes del lenguaje — el lenguaje es solo la herramienta para expresarlos.



# Arquitectura de computadoras

Para entender cómo ejecutamos programas, necesitamos saber qué hay dentro de una computadora.



# Componentes principales

- **CPU** (Unidad Central de Procesamiento): ejecuta las instrucciones. Es el "cerebro".
- **RAM** (Memoria de Acceso Aleatorio): almacenamiento temporal mientras el programa corre. Rápida pero volátil.
- **HDD / SSD** (Disco Duro / Disco Sólido): almacenamiento permanente. Aquí viven tus archivos `.py`.
- **Entrada / Salida (I/O)**: teclado, mouse, pantalla, red — cómo la computadora se comunica con el mundo exterior.

Un programa se carga desde el disco a la RAM y la CPU lo ejecuta instrucción por instrucción.

# Análisis y diseño de soluciones

Antes de escribir una sola línea de código, seguimos un proceso:

1. **Definir el problema** — ¿Qué necesito resolver exactamente?
2. **Analizar** — ¿Qué datos tengo? ¿Qué datos necesito producir?
3. **Diseñar la solución** — Describir los pasos (pseudocódigo, diagrama)
4. **Implementar** — Traducir el diseño al lenguaje de programación
5. **Probar** — Verificar que la solución funciona correctamente

El error más común: saltarse los primeros pasos y ponerse a escribir código de inmediato.

# ¿Qué es una instrucción?

Una **instrucción** es la unidad mínima de trabajo que le damos a la computadora.

Las instrucciones básicas son:

| Instrucción | ¿Qué hace?                          | Ejemplo                  |
|-------------|-------------------------------------|--------------------------|
| Leer        | Obtener datos del usuario o archivo | <code>input()</code>     |
| Escribir    | Mostrar datos en pantalla           | <code>print()</code>     |
| Calcular    | Realizar operaciones matemáticas    | <code>2 + 3</code>       |
| Decidir     | Elegir entre dos caminos            | <code>if / else</code>   |
| Repetir     | Ejecutar algo varias veces          | <code>for / while</code> |

Cualquier programa, por complejo que sea, está compuesto por estas instrucciones básicas.

# Archivos fuente

Un **archivo fuente** es un archivo de texto que contiene código escrito en un lenguaje de programación.

- En Python, los archivos fuente tienen extensión **.py**
- Son legibles por humanos (y por el intérprete)
- Se crean con cualquier editor de texto

```
mi_programa.py ← archivo fuente (código Python)
```

El código fuente describe **qué querés que la computadora haga**, en un lenguaje que vos podés entender.

# Código fuente vs. ejecutable

|             | Código fuente  | Ejecutable / Bytecode |
|-------------|----------------|-----------------------|
| Formato     | Texto legible  | Binario / bytecode    |
| Lo entiende | El programador | La CPU o la VM        |
| Extensión   | .py            | .exe , .pyc           |
| Se crea     | Con el editor  | Automáticamente       |

En Python: el intérprete convierte el `.py` en **bytecode** (`.pyc`) y lo ejecuta en la **Máquina Virtual de Python**.

# ¿Por qué Python?

- La **curva de aprendizaje** para quien nunca programó es más **suave**
- **Eficiente**: más con menos código
- Su sintaxis produce **código más limpio** y legible
- Fácil de leer, fácil de depurar
- Extensamente usado en trabajos académicos y de aplicación real: ciencia de datos, IA, automatización, web...

**Importante:** el lenguaje es una herramienta. Lo que aprendemos en esta materia aplica a cualquier lenguaje.

# ¿Por qué NO Java como primer lenguaje?

- La **curva de aprendizaje** para quien nunca programó es más **empinada**
- Hay mucha "**magia**" necesaria antes de poder hacer algo mínimo
- Arrastra **complejidades históricas** que lo hacen difícil como punto de partida
- Una vez que aprendés Python, el pasaje a Java (u otros lenguajes) es mucho más rápido

No es que Java sea malo — es que Python es mejor punto de partida.



# Tendencias de lenguajes

Python es consistentemente uno de los lenguajes más populares y de mayor crecimiento:

- **Stack Overflow Developer Survey:** Python, el lenguaje más querido por años consecutivos
- **GitHub:** uno de los lenguajes con más repositorios activos
- **RedMonk Rankings:** siempre en el top 3

La industria y la academia coinciden: Python es relevante hoy y lo seguirá siendo.

# Programamos en inglés

Los programas se escriben en inglés. Esto es importante:

- Los **nombres de funciones** y la **sintaxis** de Python están en inglés
- Las **variables** y los **comentarios** también deberían estar en inglés
- Es el idioma universal de la programación — te permite colaborar con cualquier persona en el mundo

```
# Good
student_name = "Ana"
print(student_name)

# Evitar
nombre_estudiante = "Ana"    # funciona, pero no es convención
```

# Python — Características técnicas

- **Interpretado:** no se compila a código máquina directamente
- **Alto nivel:** abstraído de los detalles del hardware
- **Propósito general:** sirve para casi cualquier tipo de problema
- **Tipado dinámico:** no declarás el tipo de las variables explícitamente
- **Garbage-collected:** la memoria se libera automáticamente
- **Multi-paradigma:** funcional, orientado a objetos, estructurado
- **Portable:** corre en Windows, Linux, macOS

Diseñado por **Guido van Rossum**. Primera versión en **1991**.

# ¿Por qué "interpretado"?

## Lenguajes compilados (C, C++):

- El código fuente se traduce **directamente a código máquina**
- El ejecutable corre directamente en la CPU

## Python (interpretado):

- El código Python se traduce a **bytecode**
- El bytecode es ejecutado por la **Máquina Virtual de Python (VM)**
- No corre directamente en la CPU — corre sobre la VM

Ventaja: más portable. Desventaja: generalmente más lento que código compilado nativo.

# Portabilidad — Platform Independent

Gracias al modelo bytecode + VM:

- El **bytecode** ( `.pyc` ) es el mismo en cualquier sistema operativo
- Siempre que la **versión del intérprete** sea la misma
- El código Python puede correr en **Windows, macOS o Linux** sin modificaciones

```
hello.py → [intérprete Python] → hello.pyc (bytecode)
                                   ↓
                               Python VM (cualquier OS)
```

"Write once, run anywhere" — siempre que haya un intérprete instalado.

# Garbage Collection — Recolección de basura

En lenguajes más antiguos (C, C++), la memoria se manejaba **manualmente**:

- El programador pedía memoria
- El programador tenía que liberarla cuando ya no la necesitaba
- Olvidarse → fuga de memoria (memory leak)

**Python lo hace automáticamente:**

- Cuando una variable deja de ser referenciada, el **Garbage Collector** libera esa memoria
- No tenés que preocuparte por esto al programar

Menos poder de control, pero menos errores comunes y más foco en resolver el problema.

# ¿Cómo funciona el intérprete de Python?

```
Archivo .py (código fuente)
    ↓
[Lexer / Parser]
    ↓
AST (Árbol sintáctico)
    ↓
[Compilador a bytecode]
    ↓
Archivo .pyc (bytecode)
    ↓
[Python Virtual Machine]
    ↓
Ejecución del programa
```

Esto ocurre automáticamente cada vez que ejecutás `python3 mi_archivo.py`.

# Resumen de la clase

| Concepto          | Idea clave                                             |
|-------------------|--------------------------------------------------------|
| Algoritmo         | Secuencia de pasos ordenados para resolver un problema |
| CPU / RAM / Disco | Los tres componentes centrales de una computadora      |
| Análisis y diseño | Pensar antes de programar                              |
| Instrucción       | La unidad mínima de trabajo para la computadora        |
| Archivo fuente    | Texto con código, extensión <code>.py</code>           |
| Python            | Interpretado, alto nivel, portable, tipado dinámico    |
| Bytecode + VM     | Cómo Python logra ser portable                         |



# Próxima clase

## Clase 1: Hello World y Variables

- Escribiremos nuestro primer programa en Python
- Aprenderemos qué son las variables y sus tipos
- Operaciones matemáticas y lógicas
- Casting entre tipos numéricos

Asegurate de tener Python 3 instalado antes de la próxima clase.

Descargalo en: [python.org/downloads](https://python.org/downloads)